

Artificial Intelligence

Prof. Chang-Hee Han

5

Fundamentals of Machine Learning

5.0 This chapter covers

▣ Chapter Preview

- Understanding the tension between generalization and optimization, the fundamental issue in machine learning
- Evaluation methods for machine learning models
- Best practices to improve model fitting
- Best practices to achieve better generalization

5.1 Generalization: The goal of machine learning

▣ Generalization: The Goal of Machine Learning

- In Chapter 4, the two examples (movie review prediction, house price regression) are split into training set, validation set, and test set.
- We do not test a model using the same data used for training — the reason becomes clear soon.
- After a few epochs, the performance on new data (never seen before) starts to get much worse than the performance on training data.
- The training performance keeps improving as training goes on.
- This means the model has started to overfit.
- Overfitting happens in all machine learning problems.

5.1 Generalization: The goal of machine learning

▣ The Core Issue in Machine Learning: Balancing Optimization and Generalization

- The main issue in machine learning is the tug-of-war between **optimization** and **generalization**.
- Optimization means adjusting the model to get the best performance possible on the training data (this is what “learning” in machine learning refers to).
- Generalization means how well the trained model performs on data it has never seen before.
- The goal, of course, is to achieve good generalization performance.
- But we cannot directly control generalization.
- We can only make the model fit the training data.
- If the model fits the training data too well, overfitting begins and generalization gets worse.

5.1 Generalization: The goal of machine learning

5.1.1 Underfitting and overfitting

- For the models you saw in the previous chapter, performance on the held-out validation data started improving as training went on and then inevitably peaked after a while.
- This pattern (illustrated in figure 5.1) is universal. You'll see it with any model type and any dataset.

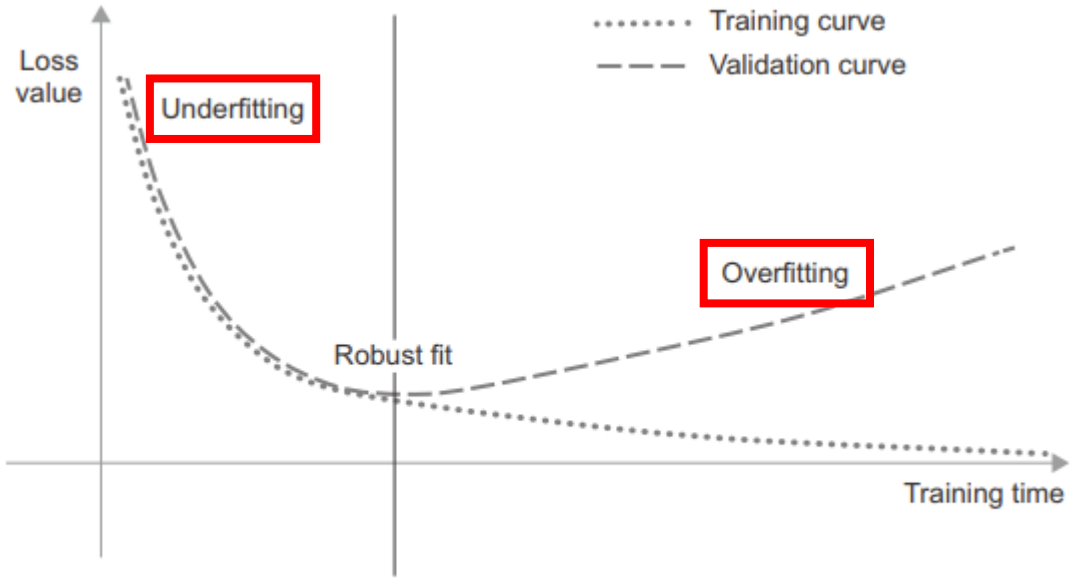


Figure 5.1 Canonical overfitting behavior

5.1 Generalization: The goal of machine learning

▣ 5.1.1 Underfitting and overfitting

- Noisy Training Data
  - ➢ Real datasets often contain wrong inputs.
  - ➢ For example, in the MNIST digits dataset, there may be images like Figure 5-2 or even completely black images.

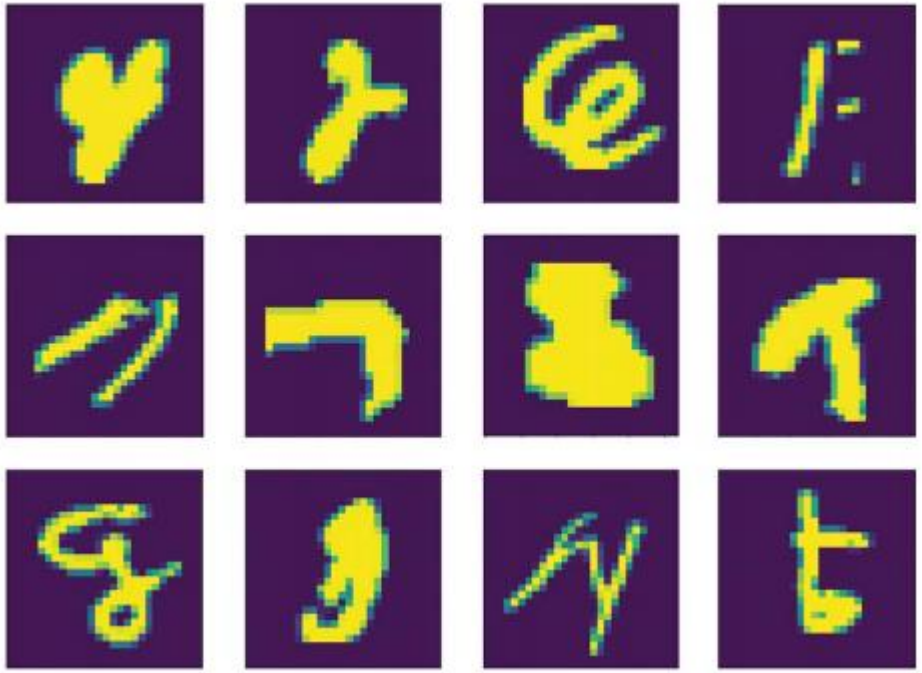


Figure 5.2 Some pretty weird MNIST training samples

5.1 Generalization: The goal of machine learning

▣ 5.1.1 Underfitting and overfitting

- Noisy Training Data
  - ➢ All of these are images from the MNIST training set.
  - ➢ Even worse, sometimes the image looks perfectly normal (like in Figure 5-3) but the label is wrong.



Figure 5.3 Mislabeled MNIST training samples

5.1 Generalization: The goal of machine learning

5.1.1 Underfitting and overfitting

- Noisy Training Data
  - If the model tries to fit these outliers, its generalization performance goes down (as in Figure 5-4).
  - For example, a digit 4 that looks very similar to the first image in Figure 5-3 might be classified as a 9.

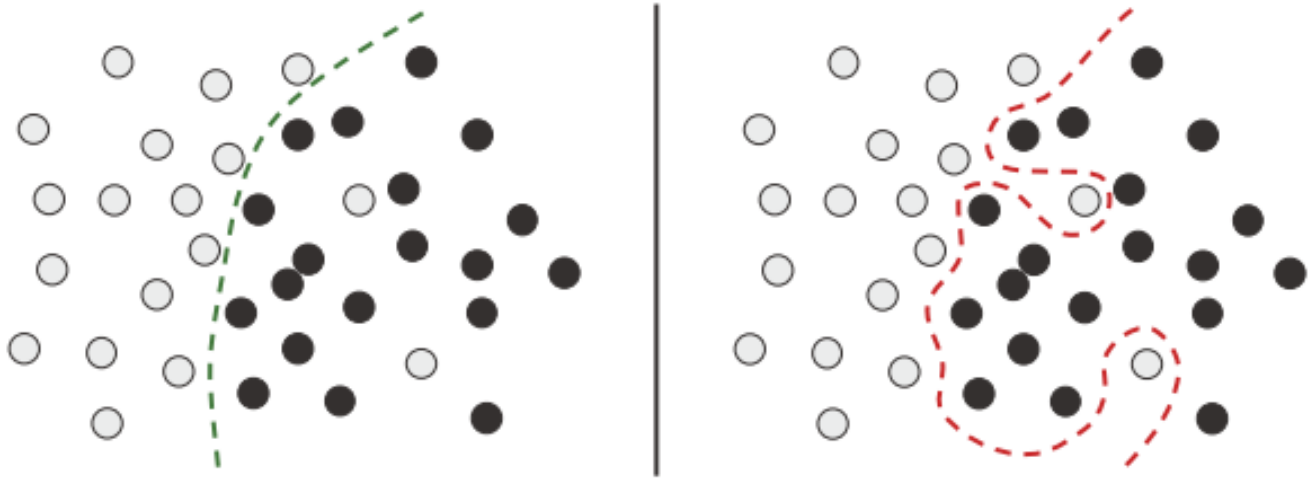


Figure 5.4 Dealing with outliers: robust fit vs. overfitting

5.1 Generalization: The goal of machine learning

▣ 5.1.1 Underfitting and overfitting

- Data Noise and Uncertainty
  - ➢ Noise is not always caused by errors — uncertainty and ambiguity can also create noise.
  - ➢ Some inputs can belong to multiple classes at once (no clear boundary).
  - ➢ Example: labeling bananas as unripe vs. ripe may differ by person.
  - ➢ Many problems also include randomness (e.g., same weather data → sometimes rain, sometimes clear).

5.1 Generalization: The goal of machine learning

▣ 5.1.1 Underfitting and overfitting

- AMBIGUOUS FEATURES
  - ➢ If a model is too confident in ambiguous areas of the feature space (like in Figure 5-5), it may overfit to such probabilistic data.
  - ➢ The best fit means ignoring individual data points and focusing on the bigger picture.

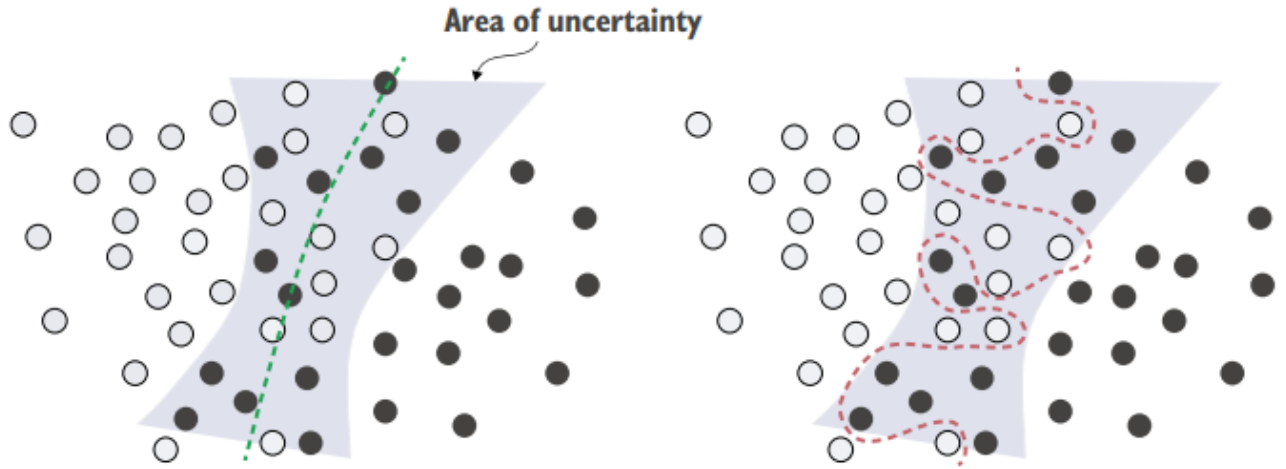


Figure 5.5 Robust fit vs. overfitting giving an ambiguous area of the feature space

5.1 Generalization: The goal of machine learning

▣ 5.1.1 Underfitting and overfitting

- Let's look at a clear example using MNIST
- In MNIST, we make new data by adding extra features with random noise or just zeros.
- These added features are meaningless and do not change the original information.
- As a result, human accuracy in classification stays the same.

Listing 5.1 Adding white noise channels or all-zeros channels to MNIST

```
from tensorflow.keras.datasets import mnist
import numpy as np

(train_images, train_labels), _ = mnist.load_data()
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype("float32") / 255

train_images_with_noise_channels = np.concatenate(
    [train_images, np.random.random((len(train_images), 784))], axis=1)

train_images_with_zeros_channels = np.concatenate(
    [train_images, np.zeros((len(train_images), 784))], axis=1)
```

5.1 Generalization: The goal of machine learning

5.1.1 Underfitting and overfitting

- Let's train two models with these two training sets.

Listing 5.2 Training the same model on MNIST data with noise channels or all-zero channels

```
from tensorflow import keras
from tensorflow.keras import layers

def get_model():
    model = keras.Sequential([
        layers.Dense(512, activation="relu"),
        layers.Dense(10, activation="softmax")
    ])
    model.compile(optimizer="rmsprop",
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])
    return model

model = get_model()
history_noise = model.fit(
    train_images_with_noise_channels, train_labels,
    epochs=10,
    batch_size=128,
    validation_split=0.2)

model = get_model()
history_zeros = model.fit(
    train_images_with_zeros_channels, train_labels,
    epochs=10,
    batch_size=128,
    validation_split=0.2)
```

5.1 Generalization: The goal of machine learning

▣ 5.1.1 Underfitting and overfitting

- Let's compare how the validation accuracy of each model evolves over time.
- Although the data has the same information, models trained with noisy data show lower accuracy because of false correlations, and more noise makes the accuracy drop further.

Listing 5.3 Plotting a validation accuracy comparison

```
import matplotlib.pyplot as plt
val_acc_noise = history_noise.history["val_accuracy"]
val_acc_zeros = history_zeros.history["val_accuracy"]
epochs = range(1, 11)
plt.plot(epochs, val_acc_noise, "b-",
         label="Validation accuracy with noise channels")
plt.plot(epochs, val_acc_zeros, "b--",
         label="Validation accuracy with zeros channels")
plt.title("Effect of noise channels on validation accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
```

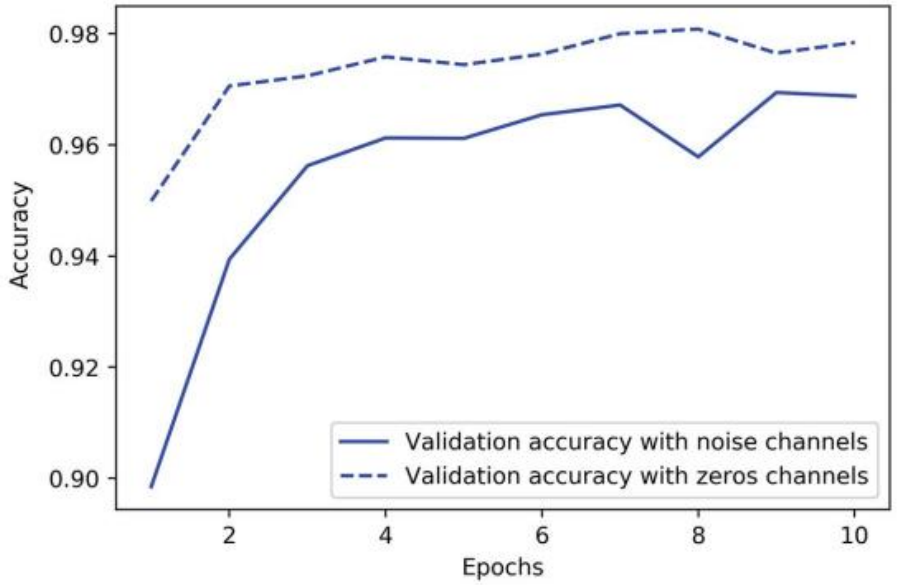


Figure 5.6 Effect of noise channels on validation accuracy

5.1 Generalization: The goal of machine learning

▣ 5.1.2 The nature of generalization in deep learning



Figure 5.10 Going from a random model to an overfit model, and achieving a robust fit as an intermediate state

5.2 Evaluating machine learning models

▣ Evaluating machine learning models

- We can only control what we can observe.
- Since the goal is to build models that generalize well to new data, we must measure generalization performance in a reliable way.

5.2 Evaluating machine learning models

▣ 5.2.1 Training, validation, and test sets

- The key idea in model evaluation is to always split the data into three sets: **training**, **validation**, and **test**.
- We train the model on the training set and check it on the validation set.
- When the model is ready for release, we test it once on the test set.
- The test data should be similar to the real-world data.
- Then we deploy the model in the real environment.
- What if we use only two sets: training and test?
- Then we just train on the training set and test on the test set — much simpler!

5.2 Evaluating machine learning models

▣ 5.2.1 Training, validation, and test sets

- The reason we don't just use training and test sets is because we always need to tune the model settings.
- For example, we may choose the number of layers or units.
- These are called hyperparameters, to distinguish them from model weights.
- We check performance on the validation set to decide these settings.
- In fact, this tuning is also a kind of learning: searching for good settings in the hyperparameter space.
- But if we keep tuning based on validation performance, the model can become overfitted to the validation set, even without being trained directly on it.

5.2 Evaluating machine learning models

▣ 5.2.1 Training, validation, and test sets

- The core idea here is **information leak**.
- Each time we tune hyperparameters based on validation performance, some information from the validation set leaks into the model.
- If we tune only once, the leak is very small.
- In that case, the validation set can still be used for fair evaluation.
- But if we tune and re-test many times, the model is exposed to too much information from the validation set.

5.2 Evaluating machine learning models

▣ 5.2.1 Training, validation, and test sets

- Splitting data into training, validation, and test sets looks simple, but when data is small, advanced methods can help.
- We will look at three common evaluation methods: **hold-out validation**, **K-fold cross-validation**, and **iterated K-fold cross-validation with shuffling**.
- We will also explain how to use a simple baseline model to check if training is going well.

5.2 Evaluating machine learning models

5.2.1 Training, validation, and test sets

- **SIMPLE HOLDOUT VALIDATION**
  - Set aside part of the data as a test set.
  - Train on the rest and evaluate on the test set.
  - As explained earlier, do not tune the model using the test set, to avoid information leak.
  - For this reason, we also need a separate validation set.

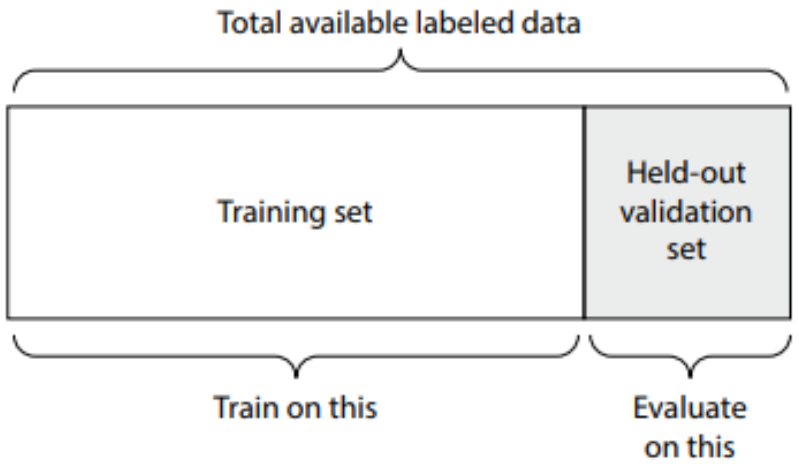


Figure 5.12 Simple holdout validation split

5.2 Evaluating machine learning models

5.2.1 Training, validation, and test sets

- **SIMPLE HOLDOUT VALIDATION**

Listing 5.5 Holdout validation (note that labels are omitted for simplicity)

```
num_validation_samples = 10000
np.random.shuffle(data)
validation_data = data[:num_validation_samples]
training_data = data[num_validation_samples:]
model = get_model()
model.fit(training_data, ...)
validation_score = model.evaluate(validation_data, ...)

...

model = get_model()
model.fit(np.concatenate([training_data,
                          validation_data]), ...)
test_score = model.evaluate(test_data, ...)
```

Shuffling the data is usually appropriate.

Defines the validation set

Defines the training set

Trains a model on the training data, and evaluates it on the validation data

At this point you can tune your model, retrain it, evaluate it, tune it again.

Once you've tuned your hyperparameters, it's common to train your final model from scratch on all non-test data available.

5.2 Evaluating machine learning models

▣ 5.2.1 Training, validation, and test sets

- **SIMPLE HOLDOUT VALIDATION**
  - ➢ This evaluation method is simple but has one drawback.
  - ➢ **When data is small, the validation and test sets may not represent the whole dataset well.**
  - ➢ You can see this easily: if shuffling with a different random seed gives very different results, this is the problem.
  - ➢ Later, we will see that K-fold cross-validation and iterated K-fold cross-validation can solve this issue.

5.2 Evaluating machine learning models

▣ 5.2.1 Training, validation, and test sets

- **K-FOLD VALIDATION**
  - In this method, the data is split into K equal parts.
  - For each part i, train the model on the other K–1 parts and test on part i.
  - The final score is the average of the K scores.
  - This helps when model performance changes a lot depending on how the data is split.
  - Like hold-out validation, this method also uses a separate validation set for tuning.

5.2 Evaluating machine learning models

▣ 5.2.1 Training, validation, and test sets

- **K-FOLD VALIDATION**

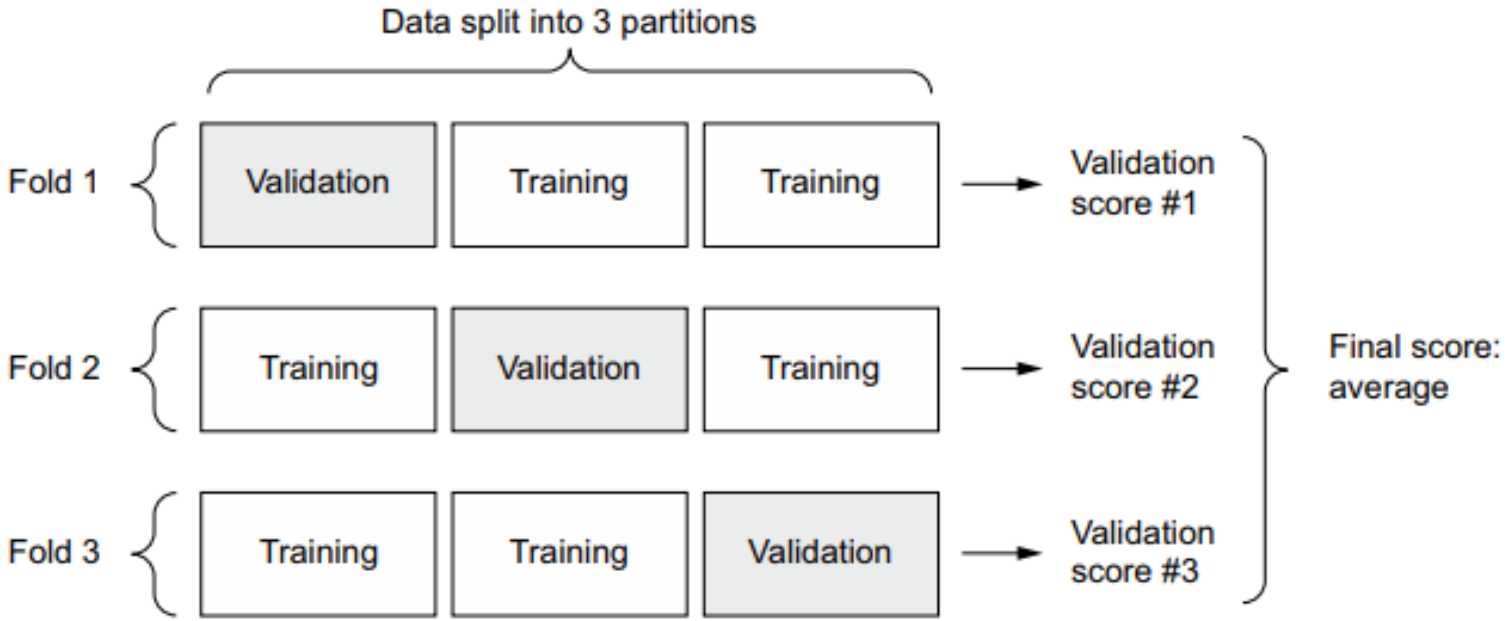


Figure 5.13 K-fold cross-validation with K=3

5.2 Evaluating machine learning models

5.2.1 Training, validation, and test sets

- **K-FOLD VALIDATION**

Listing 5.6 K-fold cross-validation (note that labels are omitted for simplicity)

```
k = 3
num_validation_samples = len(data) // k
np.random.shuffle(data)
validation_scores = []
for fold in range(k):
    validation_data = data[num_validation_samples * fold:
                           num_validation_samples * (fold + 1)]
    training_data = np.concatenate(
        data[:num_validation_samples * fold],
        data[num_validation_samples * (fold + 1):])
    model = get_model()
    model.fit(training_data, ...)
    validation_score = model.evaluate(validation_data, ...)
    validation_scores.append(validation_score)
validation_score = np.average(validation_scores)
model = get_model()
model.fit(data, ...)
test_score = model.evaluate(test_data, ...)
```

Selects the validation-data partition

Creates a brand-new instance of the model (untrained)

Validation score: average of the validation scores of the k folds

Trains the final model on all non-test data available

Uses the remainder of the data as training data. Note that the + operator represents list concatenation, not summation.

5.2 Evaluating machine learning models

▣ 5.2.2 Beating a common-sense baseline

- For MNIST digit classification, a simple baseline is validation accuracy higher than 0.1 (the score of random guessing).
- For IMDB, the baseline is accuracy higher than 0.5.
- In a binary task where 90% of samples are class A and 10% are class B, always predicting A gives 0.9 accuracy.
- A good model must perform better than such baselines.
- Having a common-sense baseline is essential when solving a problem that has not been solved before.
- If a model cannot beat a simple solution, it is not useful.
- It may mean the model is wrong, or that the problem cannot be solved with machine learning.

5.3 Improving model fit

▣ Improving model fit

- To get an optimal model, it must first be overfitted.
- Since we don't know the boundary in advance, we need to cross it to find it.
- At the start, the goal is to get a model that shows some generalization and can also overfit.
- After that, the focus is on fighting overfitting and improving generalization.

5.3 Improving model fit

▣ Improving model fit

- At this stage, three common problems can appear:
  - (1) **No training:** the training loss does not go down over time.
  - (2) **Poor generalization:** training starts well but the model cannot beat even a simple baseline.
  - (3) **Underfitting:** both training and validation loss go down and beat the baseline, but the model still does not overfit and remains too simple.
- To solve these issues, the first big milestone in a machine learning project is to get a model that beats a simple baseline, shows some generalization, and can also overfit.

5.3 Improving model fit

▣ 5.3.1 Tuning key gradient descent parameters

- Sometimes training does not start or stops too early.
- Then the loss stays flat.
- This problem can always be fixed.
- Remember: even random data can be trained by a model.
- Even if the task has no meaning, the model can still memorize the training data.
- If this happens, the problem is always in the gradient descent settings.
- Examples: optimizer choice, weight initialization, learning rate, batch size.
- All these parameters affect each other.
- In practice, it is usually enough to fix the others and just tune the learning rate and batch size.

5.3 Improving model fit

5.3.1 Tuning key gradient descent parameters

- Let's look at a concrete example.
- Train two MNIST models with an improperly large learning rate of 1.0.

Listing 5.7 Training an MNIST model with an incorrectly high learning rate

```
(train_images, train_labels), _ = mnist.load_data()
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype("float32") / 255

model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax")
])
model.compile(optimizer=keras.optimizers.RMSprop(1.),
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_images, train_labels,
          epochs=10,
          batch_size=128,
          validation_split=0.2)
```



5.3 Improving model fit

5.3.1 Tuning key gradient descent parameters

- This model quickly reaches about 30–40% training and validation accuracy, but cannot go higher.
- Let's lower the learning rate to a more reasonable value, 1e-2.

Listing 5.8 The same model with a more appropriate learning rate

```
model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax")
])
model.compile(optimizer=keras.optimizers.RMSprop(1e-2),
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_images, train_labels,
          epochs=10,
          batch_size=128,
          validation_split=0.2)
```

5.3 Improving model fit

▣ 5.3.2 Leveraging better architecture priors

- Sometimes the model trains, but the validation score does not improve at all.
- It performs no better than a random classifier.
- In other words, the model trains but does not generalize.
- This is one of the worst situations in machine learning.
- It means something is fundamentally wrong in the approach, though it may be hard to know what.
- A few tips:
  - The input data may not have enough information to predict the target.
  - In that case, the problem cannot be solved this way.
- We saw this earlier when training MNIST with shuffled labels:
  - The model trained, but validation accuracy stayed at 10%.
  - Such a dataset cannot generalize by design.

5.3 Improving model fit

▣ 5.3.2 Leveraging better architecture priors

- The type of model we use may not fit the problem.
- For example, in Chapter 10 we will see a timeseries prediction task with a dense neural network.
- Such a structure cannot beat a simple baseline.
- A recurrent neural network (RNN) is more suitable and generalizes better.
- To generalize, we need a model that makes the right assumptions about the problem.
- In short, we must choose the right structure for the task.

5.3 Improving model fit

▣ 5.3.3 Increasing model capacity

- Next, we want the model to overfit.
- For example, think of a small model trained on MNIST pixels — a simple logistic regression.

Listing 5.9 A simple logistic regression on MNIST

```
model = keras.Sequential([layers.Dense(10, activation="softmax")])
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
history_small_model = model.fit(
    train_images, train_labels,
    epochs=20,
    batch_size=128,
    validation_split=0.2)
```

5.3 Improving model fit

▣ 5.3.3 Increasing model capacity

- You get loss curves that look like figure 5.14:

```
import matplotlib.pyplot as plt
val_loss = history_small_model.history["val_loss"]
epochs = range(1, 21)
plt.plot(epochs, val_loss, "b--",
         label="Validation loss")
plt.title("Effect of insufficient model capacity on validation loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
```

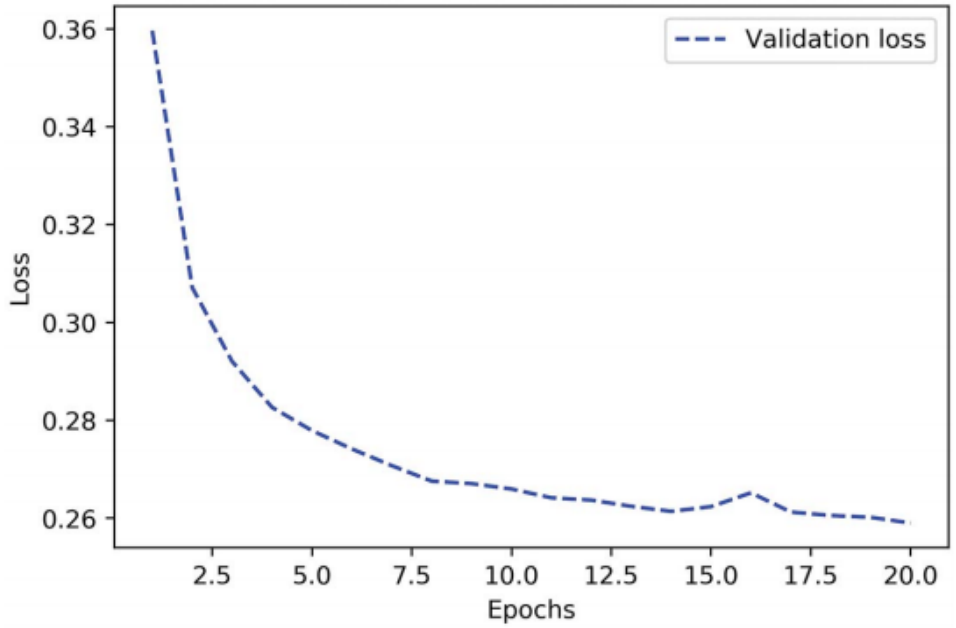


Figure 5.14 Effect of insufficient model capacity on loss curves

5.3 Improving model fit

5.3.3 Increasing model capacity

- The validation loss reaches a point and then stops improving, or improves very slowly.
- Even after many passes on the training data, the model does not overfit.
- Sometimes our careers feel like this too!
- Remember: overfitting is always possible.
- Like with training loss that does not decrease, this problem can also be solved.
- If the model cannot overfit, it means its representational power is too low.
- We need a larger model that can store more information.
- Solutions:
  - Add more layers.
  - Make layers bigger (more weights).
  - Use a different type of layer that fits the problem better.

5.3 Improving model fit

▣ 5.3.3 Increasing model capacity

- Let's try training a bigger model, one with two intermediate layers with 96 units each:

```
model = keras.Sequential([
    layers.Dense(96, activation="relu"),
    layers.Dense(96, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
history_large_model = model.fit(
    train_images, train_labels,
    epochs=20,
    batch_size=128,
    validation_split=0.2)
```

5.3 Improving model fit

▣ 5.3.3 Increasing model capacity

- The validation curve now looks exactly like it should: the model fits fast and starts overfitting after 8 epochs (see figure 5.15).

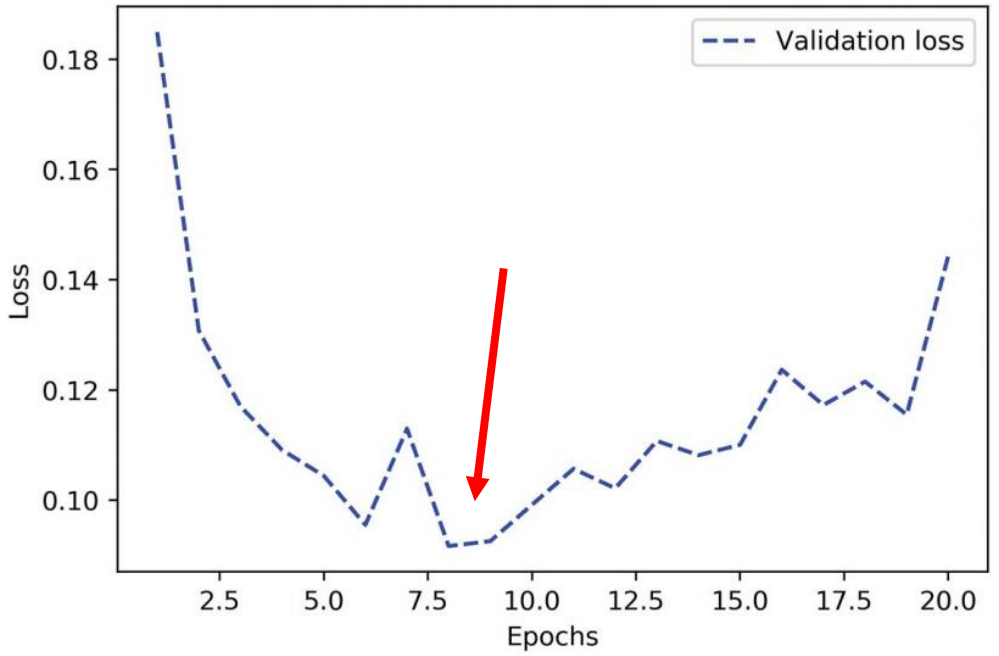


Figure 5.15 Validation loss for a model with appropriate capacity

5.4 Improving generalization

▣ 5.4.1 Dataset curation

- Deep learning is curve fitting, not magic.
- Generalization comes from the hidden structure of data.
- Good data is more important than a better model.
- Spending time and money on data collection and cleaning gives higher return than just improving the model.
- **Good models start with good data.**

5.4 Improving generalization

▣ 5.4.1 Dataset curation

- **How to Prepare a Good Dataset**
  - ➢ **(1) Have enough data**
    - Need dense sampling of input–output space
    - More data → better model
  - ➢ **(2) Reduce labeling errors**
    - Check inputs for mistakes or strange values
    - Proofread and correct labels
  - ➢ **(3) Clean the data**
    - Handle missing values carefully
  - ➢ **(4) Feature selection**
    - Remove features that are not useful

5.4 Improving generalization

▣ 5.4.2 Feature engineering

- Feature engineering
  - ➢ the step where we use knowledge about data and machine learning algorithms (here, neural networks).
- Before giving data to the model, we apply hand-coded transformations (not learned) so the algorithm can work better.
- In many cases, it is hard to expect a machine learning model to learn perfectly from raw data.
- Data needs to be expressed in a way that makes the model's work easier.

5.4 Improving generalization

▣ 5.4.2 Feature engineering

- Example:
  - ➢ Suppose we want to build a model that takes a clock image as input and outputs the time of day.
  - ➢ If we only use the raw pixels of the image as input, this becomes a very hard machine learning problem.
  - ➢ By using our knowledge of how to read a clock, we can create better features (like hand coordinates or angles), making the problem so simple that machine learning is barely needed.

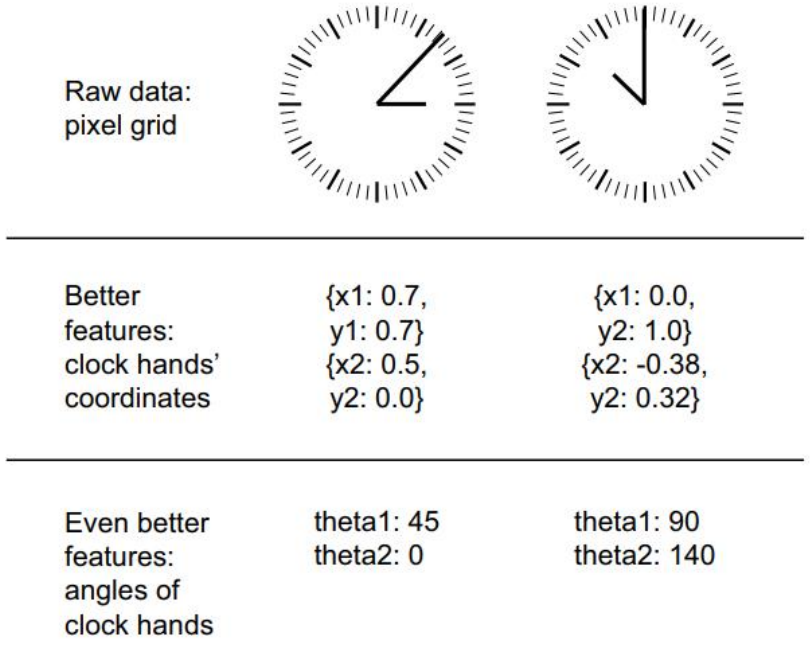


Figure 5.16 Feature engineering for reading the time on a clock

5.4 Improving generalization

▣ 5.4.2 Feature engineering

- This is the core of feature engineering.
- Express features in a simpler way to make the problem easier.
- Make the latent manifold smoother, simpler, and more structured.
- To do this, you usually need a deep understanding of the problem.
- Before deep learning, **feature engineering** was the most important part of the machine learning workflow.

5.4 Improving generalization

▣ 5.4.2 Feature engineering

- Fortunately, modern deep learning usually does not need much feature engineering.
- Neural networks can automatically extract useful features from raw data.
- Then, do we not need to care about feature engineering when using deep networks?
- Not exactly — for two reasons:
  - ➢ (1) Good features can solve problems more efficiently with fewer resources.
    - Example: Using a CNN to read clock hands is not the best choice.
  - ➢ (2) Good features allow models to work with less data.
    - Deep models learn features well only when there is a lot of training data.
    - When samples are few, the information in features becomes very important.

5.4 Improving generalization

▣ 5.4.3 Using early stopping

- Deep learning models usually have too many parameters.
- This means they have much more freedom than the minimum needed to learn the latent manifold.
- But this is not a problem, because deep models are not trained until the end.
- If trained fully, the model would not generalize at all.
- Training is always stopped before the loss reaches its minimum.
- Finding the exact point during training where generalization is best (the balance between underfitting and overfitting) is one of the most effective ways to improve performance.

5.4 Improving generalization

▣ 5.4.3 Using early stopping

- In the earlier example, the model was trained longer than needed to find the epoch with the best validation score.
- Then, a new model was trained again for exactly that number of epochs.
- This works, but it is repetitive and often costly.
- Instead, you can save the model at the end of each epoch, then reuse the one with the best score.
- In Keras, this is usually handled with the **EarlyStopping callback**.
- Training stops when validation no longer improves, and the model with the best validation score so far is kept.

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **Regularization** methods are best practices that limit a model's ability to fit the training data too perfectly.
- The goal is to improve validation scores.
- Regularization makes models simpler, smoother, and more general.
- By preventing overfitting to the training set, the model can better approximate the latent manifold of the data, improving generalization.
- Always remember: regularization requires a careful evaluation process.
- Generalization is possible only when it can be measured.

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **REDUCING THE NETWORK'S SIZE**
  - ➢ Let's look at the most common regularization method and apply it to the movie classification model from Chapter 4 to improve performance.
  - ➢ The simplest way to reduce overfitting is to reduce the model size (number of layers and units → number of trainable parameters).
  - ➢ With limited memory capacity, the model cannot just memorize the training data.
  - ➢ Instead, it learns compressed representations that predict the target well.
  - ➢ These compressed representations are exactly what we want.
  - ➢ At the same time, the model must have enough parameters to avoid underfitting.
  - ➢ The model should not have too little capacity.
  - ➢ We need to find a balance between too much and too little capacity.

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **REDUCING THE NETWORK'S SIZE**
  - ➢ Unfortunately, there is no magic formula to decide the right number of layers or units per layer.
  - ➢ To find the best model size, you must test different structures (on the validation set, not the test set).
  - ➢ A common workflow to find the right size is:
    - (1) Start with fewer layers and parameters.
    - (2) Then, increase the number of layers or units until validation loss stops improving.

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **REDUCING THE NETWORK'S SIZE**
  - ➢ Let's try this on the movie-review classification model. The following listing shows
  - ➢ our original model.

Listing 5.10 Original model

```
from tensorflow.keras.datasets import imdb
(train_data, train_labels), _ = imdb.load_data(num_words=10000)

def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))
    for i, sequence in enumerate(sequences):
        results[i, sequence] = 1.
    return results
train_data = vectorize_sequences(train_data)
model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
history_original = model.fit(train_data, train_labels,
                             epochs=20, batch_size=512, validation_split=0.4)
```

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **REDUCING THE NETWORK'S SIZE**

  ➢ Now let's try to replace it with this smaller model.

Listing 5.11 Version of the model with lower capacity

```
model = keras.Sequential([
    layers.Dense(4, activation="relu"),
    layers.Dense(4, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
history_smaller_model = model.fit(
    train_data, train_labels,
    epochs=20, batch_size=512, validation_split=0.4)
```

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **REDUCING THE NETWORK'S SIZE**

  - Figure 5.17 shows a comparison of the validation losses of the original model and the smaller model.

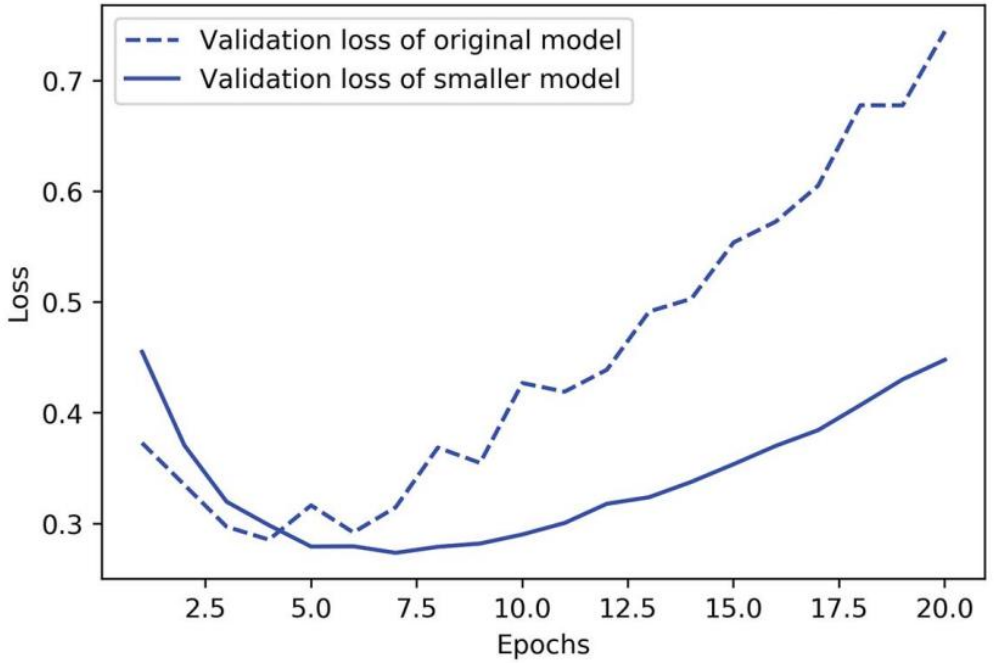


Figure 5.17 Original model vs. smaller model on IMDB review classification

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **REDUCING THE NETWORK'S SIZE**
  - ➢ Now let's compare a network with much larger capacity than needed for the problem.
  - ➢ It is common to build models with too many parameters, but memory capacity can become too large.
  - ➢ If the model quickly overfits and the validation loss curve is unstable and noisy, the model is too big.
  - ➢ (Unstable validation metrics may also mean the validation process is not reliable — for example, if the validation set is too small.)

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **REDUCING THE NETWORK'S SIZE**

Listing 5.12 Version of the model with higher capacity

```
model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(512, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
history_larger_model = model.fit(
    train_data, train_labels,
    epochs=20, batch_size=512, validation_split=0.4)
```

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **REDUCING THE NETWORK'S SIZE**

  - Figure 5.18 shows how the bigger model fares compared with the reference model.

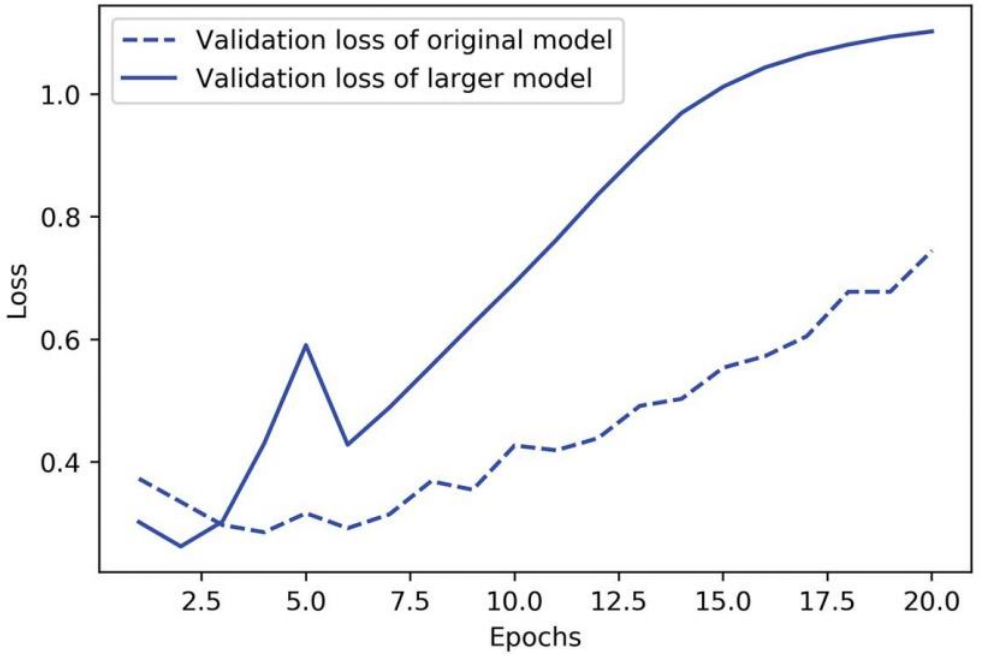


Figure 5.18 Original model vs. much larger model on IMDB review classification

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **ADDING WEIGHT REGULARIZATION**
  - For given training data and a network structure, there can be many sets of weights (models) that explain the data.
  - A simpler model is usually less likely to overfit than a complex one.
  - Here, a simple model means one with low entropy in parameter values (or fewer parameters, as seen earlier).
  - A common way to reduce overfitting is to limit model complexity so that weights stay small.
  - This makes the distribution of weights more balanced.
  - This method is called **weight regularization**, where we add a cost to the loss function for large weights.

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **ADDING WEIGHT REGULARIZATION**

  - There are two types of regularization costs:

    - **L1 regularization:**

      ✓ Adds a cost proportional to the absolute value of weights (L1 norm).

    - **L2 regularization:**

      ✓ Adds a cost proportional to the square of weights (L2 norm).

  - In neural networks, L2 regularization is also called weight decay.

  - The names are different, but don't be confused — weight decay is mathematically the same as L2 regularization.

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **ADDING WEIGHT REGULARIZATION**

Listing 5.13 Adding L2 weight regularization to the model

```
from tensorflow.keras import regularizers
model = keras.Sequential([
    layers.Dense(16,
                 kernel_regularizer=regularizers.l2(0.002),
                 activation="relu"),
    layers.Dense(16,
                 kernel_regularizer=regularizers.l2(0.002),
                 activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
history_l2_reg = model.fit(
    train_data, train_labels,
    epochs=20, batch_size=512, validation_split=0.4)
```

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **ADDING WEIGHT REGULARIZATION**
  - ➢ Figure 5.19 shows the impact of the L2 regularization penalty.
  - ➢ As you can see, the model with L2 regularization has become much more resistant to overfitting than the reference model, even though both models have the same number of parameters.

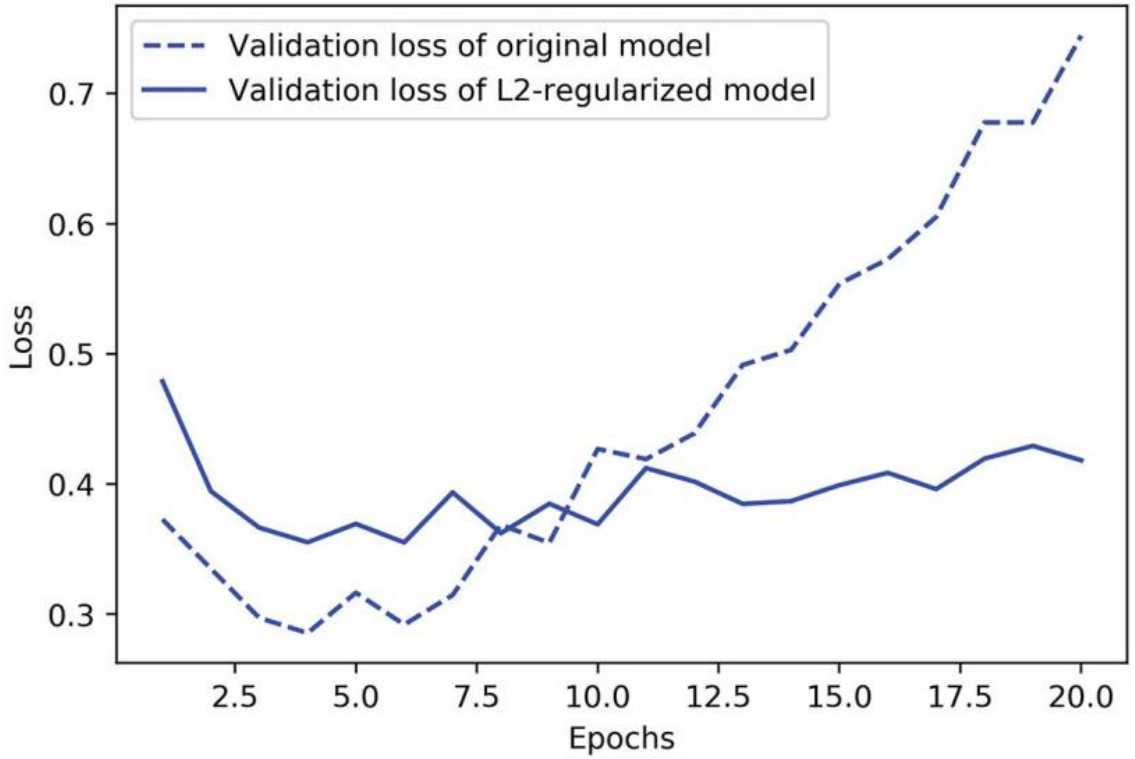


Figure 5.19 Effect of L2 weight regularization on validation loss

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **ADDING WEIGHT REGULARIZATION**

  ➢ As an alternative to L2 regularization, you can use one of the following Keras weight regularizers.

Listing 5.14 Different weight regularizers available in Keras

```
from tensorflow.keras import regularizers
regularizers.l1(0.001)
regularizers.l1_l2(l1=0.001, l2=0.001)
```

L1 regularization

Simultaneous L1 and L2 regularization

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **ADDING DROPOUT**
  - ➢ Dropout is one of the most effective and most commonly used regularization techniques for neural networks
  - ➢ **Dropout**, applied to a layer, consists of randomly dropping out (setting to zero) a number of output features of the layer during training.
    - Let's say a given layer would normally return a vector [0.2, 0.5, 1.3, 0.8, 1.1] for a given input sample during training.
    - After applying dropout, this vector will have a few zero entries distributed at random:
      - ✓ for example, [**0**, 0.5, 1.3, **0**, 1.1].
  - ➢ The dropout rate is the fraction of the features that are zeroed out;
    - it's usually set between 0.2 and 0.5.

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **ADDING DROPOUT**

  - Note that this process can be implemented by doing both operations at training time and leaving the output unchanged at test time, which is often the way it's implemented in practice (see figure 5.20):

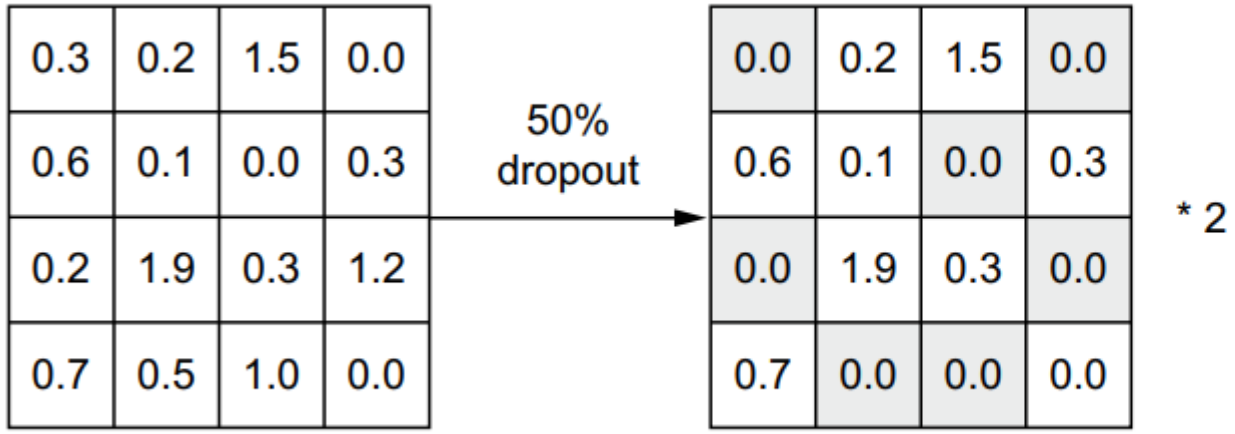


Figure 5.20 Dropout applied to an activation matrix at training time, with rescaling happening during training. At test time the activation matrix is unchanged.

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **ADDING DROPOUT**
  - In Keras, you can introduce dropout in a model via the Dropout layer, which is applied to the output of the layer right before it.
  - Let's add two Dropout layers in the IMDB model to see how well they do at reducing overfitting.

Listing 5.15 Adding dropout to the IMDB model

```
model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(16, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
history_dropout = model.fit(
    train_data, train_labels,
    epochs=20, batch_size=512, validation_split=0.4)
```

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **ADDING DROPOUT**
  - ➢ Figure 5.21 shows a plot of the results.
  - ➢ This is a clear improvement over the reference model—it also seems to be working much better than L2 regularization, since the lowest validation loss reached has improved.

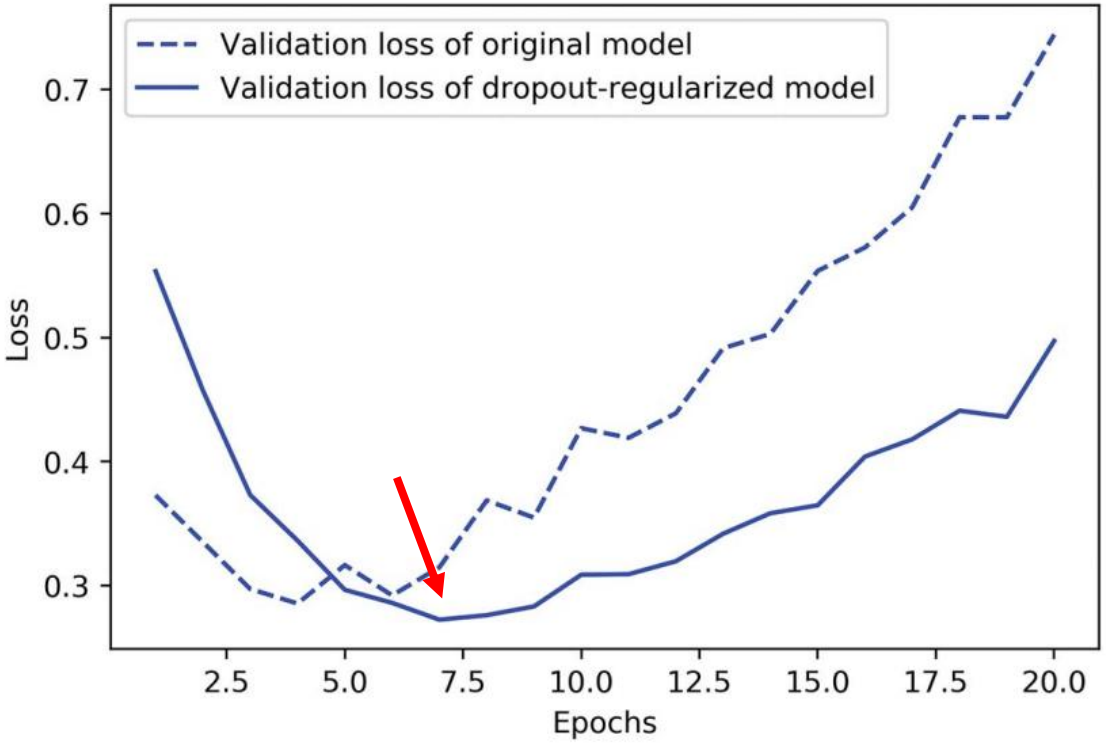


Figure 5.21 Effect of dropout on validation loss

5.4 Improving generalization

▣ 5.4.4 Regularizing your model

- **To recap, these are the most common ways to maximize generalization and prevent overfitting in neural networks:**
  - ➢ Get more training data, or better training data.
  - ➢ Develop better features.
  - ➢ Reduce the capacity of the model.
  - ➢ Add weight regularization (for smaller models).
  - ➢ Add dropout.



Q & A

Prof. Chang-Hee Han